CSC8112 - Internet of Things

Assessment 1

Assessment Overview

This assessment contributes 40% towards the total mark for this module. Out of this, 80% marks are assigned for implementation tasks and 20% marks are for the final report. It is an individual exercise: no group work is permitted for the assessment. You are advised to read and view all the instructional tutorial resources before you start implementing the solutions for the coursework Tasks 1 to 5. Each Task has been assigned a specific mark, which you will be awarded once you successfully demonstrate the completion of the same.

Once you complete Tasks 1-5, you will need to prepare the final Report (Task 6). **This coursework Report must be submitted on NESS by 3 pm on November 20, 2026**. In this Report, you will need to provide an in-depth discussion of how you implemented the solutions (e.g., code and commands) to solve Tasks 1-5. Additionally, you will be required to demonstrate successful executions of Tasks 1-5. Before the report submission deadline, you will be provided with a 30 mins slot to conduct **live demonstration**. In case of unforeseen disruptions (e.g., further lockdowns), we may also allow recorded demonstrations.

Final marks for this coursework will be decided by your performance in the live demonstrations and the technical details you will provide in the final report. While the final report needs to be submitted to NESS by 3 pm on November 20, 2026, the live demonstration session will be organised on November 16, 2026.

Objectives

The learning outcomes of this coursework include the following:

- Understand how to process Internet of Things (IoT) sensor data in the edge-cloud setting?
- Be able to develop a machine learning-based IoT data processing pipeline (data collection, data preprocessing, classification, prediction, object detection, and visualisation) in the edge-cloud setting?
- Be able to use a lightweight virtualisation technology stack, such as Docker, to implement IoT data processing pipeline in the edge-cloud setting?

Motivation

Newcastle City Council wants to know whether the busiest roads in the city also have the worst air quality, but streaming every sensor reading and every camera frame to the cloud would be slow and costly. Instead, the work is done at the edge. A data collector pulls live PM2.5 readings and camera frames from the Urban Observatory, and the Mosquitto broker passes them to separate microservices. One counts vehicles in each frame, one classifies the air quality level, and one predicts future PM2.5 values. The decision module combines the vehicle counts with the air-quality results and sends only the frames showing heavy traffic, with the latest air-quality reading, to the cloud dashboard. Your job is to build and connect these services, packaged in Docker, so that the dashboard lets the council compare traffic and air quality side by side.

System concept

After successfully completing the coursework, you will be able to gain hands-on experience in the following interrelated technology stacks, including:

- configuring a Docker-based IoT data processing pipeline;
- pulling images from the Docker Hub (a global repository of software components' images maintained by the developers);
- creating and deploying a self-contained IoT data processing service, which is often referred to as microservices;
- training a machine-learning-based predictor based on real-world data streams available from Newcastle Urban Observatory;
- implementing a machine learning-based air-quality prediction micro-service;
- visualizing time-series data using graphs.

A high-level picture showing the overall system design scope of the coursework is shown in Figure 1, a short explanation of the components is given below:

IoT tier:

- *Newcastle Urban Observatory (NCL UO) [https://urbanobservatory.ac.uk/]*: The largest set of publicly available real-time urban data in the UK. NCL UO sensors are gathering data across Newcastle city. With over 50 data types and counting, there are lots of live data for you to access.

Edge tier:

- *Data collector*: This will be a software component that you will design and implement in Task 1, focusing on (i) reading data from Urban Observatory API and (ii) transmitting data to the machine learning pipeline.

- *Mosquitto*: A broker of MQTT protocol, a message queuing system given to you as a Docker image, which forms the basis for enabling asynchronous service-to-service communication in a complex Machine Learning (ML)-based IoT data processing pipeline.

- *Machine Learning Object Detection Model*: A microservice to detect vehicles over the camera stream images.

- *Machine Learning Classifier Model*: A microservice that uses machine learning to train from past air quality data to classify air quality level

- *Machine Learning Prediction Model*: A microservice that uses machine learning to train from air quality data and predict future PM2.5 values.

- *Visualization*: A component that will visualize the trend of raw time-series data and the prediction results (input from the Machine Learning Models).

- *Decision module*: A microservice that collects the output from object detection and the machine learning classifier model. Then, push the metrics to the cloud dashboard.

Cloud tier:

- *Cloud dashboard*: A web dashboard showing the street of when air pollution is high

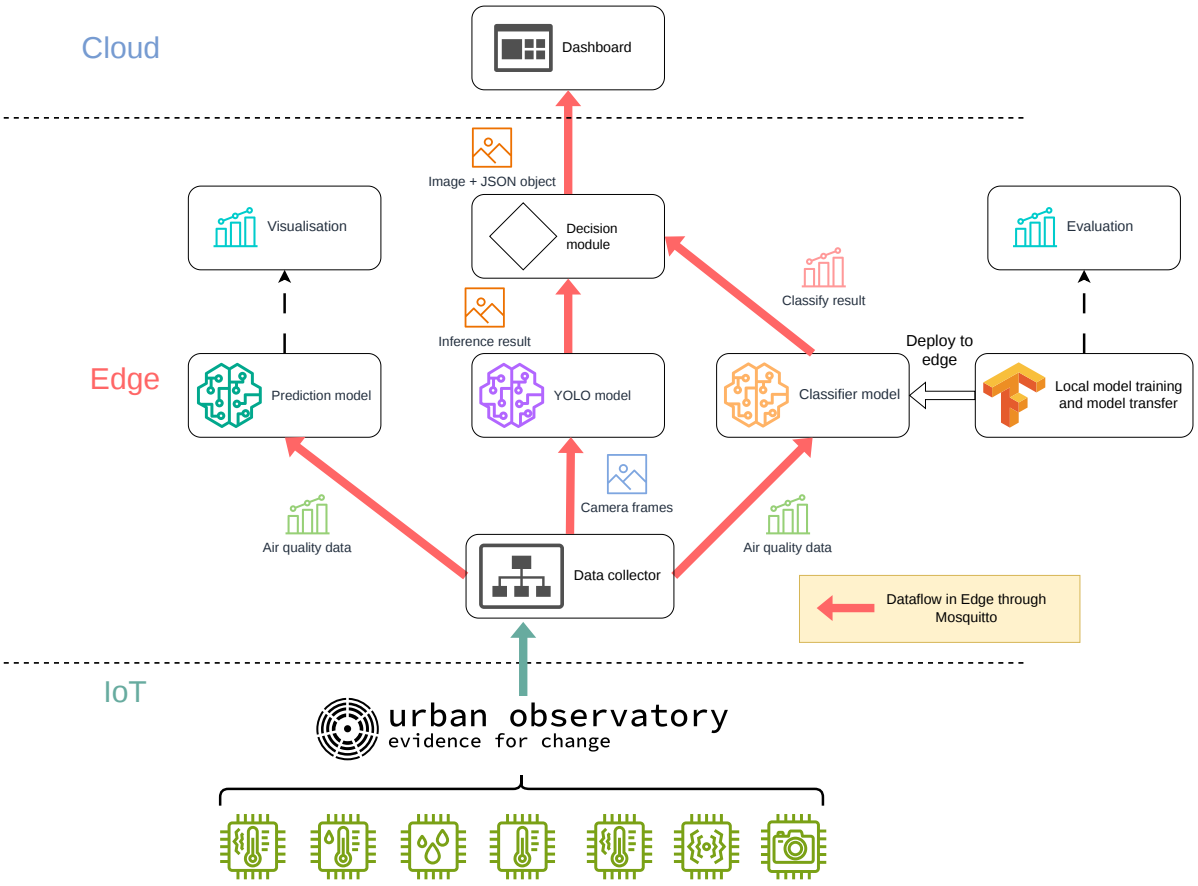


Figure 1: System overview

Pre-Requisites

Before starting the coursework, you are advised to carefully go through the training content covered in Lecture 1 and extra supplements provided at [https://github.com/ncl-iot-team/CSC8112]. Together, these provide in-depth detail on:

- how to access and get started on the Apporto platform;
- how to download and run a docker image;
- how to run your experiments;
- some hints for system structures of every task.

Specification of Tasks

The coursework consists of 6 tasks. **Please note that Tasks 1-5 need to be done by both the command line and implementing the logic using Python language.**

Task 1: Design a data collector component by leveraging Newcastle Urban Observatory IoT data streams (15 Marks)

Task Objectives: Understand and learn how to pull and run a Docker image from Docker Hub using the command line interface, how to collect real-world IoT data streams by invoking the Application Programming Interface (API) of Newcastle Urban Observatory, how to send data via MQTT (With Mosquitto as the broker for IoT applications), and how to re-compile and build Docker image using the command line and programmatic interfaces.

Hints: It is generally a good practice to use multiple topics for different data types. Large messages should be split before sending to MQTT.

1. Pull and run the Docker image "eclipse/mosquitto" from Docker Hub in the virtual machine running.
2. Develop a data collector component with the following functions (Code):
   (a) Collect sensor data by sending an HTTP GET request to `http://smartcam.uo.ncl.ac.uk:8001/sensor`. Each response is a JSON array of sensor readings over 10 minutes of real data.
   (b) Collect camera frames by sending an HTTP GET request to `http://smartcam.uo.ncl.ac.uk:8001/camera`. Each response is a single JPEG image.
   (c) Both endpoints update every 5 seconds and loops the data every 9 minutes. You must therefore poll both endpoints periodically to build up a continuous data stream.
   (d) Process the raw sensor data
      - Although the raw air quality data you collected from the Urban Observatory API contains many metrics including NO_2, NO, CO_2, PM2.5, and PM10, among others, for the purpose of this coursework you only need to store and analyze PM2.5 data. While many meta-data are available for PM2.5 data, such as sensor name, timestamp, value, and location, you only need to store the metrics related to the Timestamp and Value meta-data fields.
      - Filter out outliers (If the value is greater than 50), print out the outliers to the console
   (e) Send all PM2.5 data to be used by Task 3 via MQTT.
   (f) Send all camera frames to be used by Task 2 via MQTT
   (g) Package the data collector as a Docker image and run with *Docker compose*

Task 2: YOLO model Inference for Smart Camera (15 Marks)

Task Objectives: Understand how to process data from IoT for machine learning inference. Moreover, you will also learn how to collect/send image data using various message queuing protocols (e.g., MQTT), which are central to IoT data stream management. This task will also

help you understand how native *Docker Compose techniques* can be leveraged to manage and deploy a complex IoT application stack/pipeline for image classification problems.

1. Develop a camera inference application script that counts the number of vehicles in each camera frame
   (a) Each camera frame comes from the data collector from Task 1
   (b) Highlight the vehicles in the frame with a red rectangular box into a second image using *YOLO* detection model. You can use any YOLO version for detection like the *YOLO11n, YOLOv8n, YOLOv8*.
   (c) Send the annotated image and vehicle count to be used by Task 5 via MQTT
2. Define a Dockerfile to package the YOLO model inference as a Docker image
3. Extend your *Docker compose file* from Task 1 to sets up the camera inference application you just developed
4. Run the camera inferencing using Docker compose

Hints: An MQTT payload is bytes, not a NumPy array. Encode the YOLO annotated frame, convert the result to bytes, and publish those bytes directly as the payload.

Task 3: Time-series data prediction and visualization **(10 Marks)**

Task Objectives: Understand how to use a machine learning model/classifier with time-series sensor data, that you prepared in Task 2, to make a prediction, and how to visualize those data and predicted results.

1. Download a pre-defined Machine Learning (ML) engine code froringom [https://github.com/ncl-iot-team/CSC8112_MLEngine].
2. Design a PM2.5 prediction operator with the following functions (code):
   (a) Collect PM2.5 data computed by Task 1 from MQTT until you have 8706 data points.
   (b) Filter out any duplicate data and sort based on timestamp. (Your last 2 data should be 2.925 and 3.295)
   (c) Average the PM2.5 data over 10-minute periods.
   (d) Convert timestamp to date time format (year-month-day hour:minute:second), and please print out the PM2.5 data with the reformatted timestamp to the console.
   (e) Use the line chart component of matplotlib to visualize averaged PM2.5 daily data, directly display the figure or save it as a file.
   (f) Feed averaged PM2.5 data to machine learning model to predict the trend of PM2.5 for the next 2 hours.
   (g) Define a Dockerfile to package the prediction model.

(h) Extend your *Docker compose file* from Task 1. Create a bind mount from the container to the host.

(i) Visualize predicted results from ML predictor/classifier model, save any graphs (predefined in the provided Machine Learning code) as a file to the bind mount your created.

Hints: To install the python dependency package "matplotlib" (a data visualization tool) use the following library including [https://pypi.org/project/matplotlib/]. To download the package "prophet" (a machine learning tool) use the following link: [https://pypi.org/project/prophet/].

Task 4: Local Model Training and Edge Inference for Air Quality Classification (20 Marks)

Task Objectives: Understand how to train a classification model locally, convert it to TensorFlow Lite for edge deployment, transfer the optimized model to edge infrastructure, and implement air quality classification using historical IoT data.

1. Use the provided labeled air quality dataset with PM2.5 values [https://github.com/ncl-iot-team/CSC8112/blob/main/data/PM2.5_labelled_data.csv], and train a classification model locally in your machine or google colab or kaggle environment with cpu/gpu(optional) using TensorFlow into corresponding 3 classifications as Red, Yellow, and Green. You can perform label-encoding, normalization, sampling to the labeled data if needed to improve model quality.

2. Convert the trained model to TensorFlow Lite format (.tflite), apply quantization to reduce model size and save it.

3. Evaluate model performance (accuracy, precision, recall, confusion matrix) and visualize the model size comparison graph(original vs TFLite optimized).

4. Transfer the .tflite model and any other necessary file(if needed) using your personal github repo file URL with a "curl" command.

5. Create a Python script on Edge VM that:

   (a) Loads the TensorFlow Lite model.

   (b) Subscribe to the MQTT broker

   (c) Perform batch-inference on the json data from the MQTT broker.

   (d) Classifies each PM2.5 reading as Green/Yellow/Red using .tflite model file.

   (e) Log classification results with the PM2.5 value to console.

   (f) Send the classification results over MQTT for Task 5 to process.

   (g) Define a Dockerfile to package the classification model

   (h) Extend your *Docker compose file* from Task 1. Create a bind mount from the container to the host

(i) Visualize results with a bar-chart showing count of Green/Yellow/Red classifications, a Time-series plot of PM2.5 values with color-coded classification points, PM2.5 value frequency distribution, and a summary of classification.

Hints: You can use the keras sequential package for the classification model. No need for specific models. Just explain your decisions in the report as we're more concerned on the task completion and not on model and inference quality for this task. Distribution of PM2.5 values over the classes on IoT Sensor data from Urban Observatory is 7927 for Green, 772 for Yellow, and 12 for Red class, with total count being 8711. Here's the solution overview - https://github.com/ncl-iot-team/CSC8112/blob/main/img/task4.png

Task 5: Decision module **(20 marks)**

Task Objectives:

1. Read the PM2.5 data with the classified result from Task 4 and the YOLO annotated images from Task 2 over MQTT.
2. Send the annotated image with the latest "timestamp", "value", "predicted_quality" if the detected vehicle count from task 2 result is more than 5, using API call to the dashboard ["http://dashboard.cloud.ncl.ac.uk:8000/api/records"].
3. Define a Dockerfile to package the decision module
4. Extend your *Docker compose file* from Task 1

Hints:

- The PM2.5 timestamps are in milliseconds, but the dashboard expects Unix seconds. Convert with $ts/1000$.
- The image, the vehicle count and the PM2.5 result arrive as separate MQTT messages at different times, and none of them waits for the others. One approach is to subscribe to all three topics and keep only the most recent image and the most recent PM2.5 result in memory. The vehicle count message acts as the trigger when one arrives and is above the threshold, the module uses whatever latest image and result it holds, and skips the frame if either is still missing. You may design this differently, for example by matching messages by frame number or timestamp. Explain in your report how your module handles a late or missing stream, and the trade-off of your approach.
- You can leverage MQTTX [https://mqttx.app/docs] on the VM for visualising the MQTT messages on your topics.

Task 6: Report **(20 Marks)**

Prepare the Final Report in plain English. There is no word or page limit; however, we appreciate a clear, concise, and focused presentation style. The report should consist of:

1. Detailed response to each task and related sub-tasks.
2. Screenshots of running services in the Docker Environment.
3. Screenshots of Code Snippets and/or Docker console.
4. Plots of data and prediction results by using Matplotlib.
5. Analytical discussion of the results and related conclusions.